Argus: A Channel Interface for Public-Coin Interactive Protocols

Ricardo Perello

Final project report, June 2026

Abstract

Argus is a Rust research prototype for writing public-coin interactive protocols once and executing them through multiple backends. Its central design choice is to express protocol logic as a channel program: the prover sends messages and reads public verifier messages, while the verifier reads prover messages and sends public verifier messages. The backend, not the protocol, owns the transcript. This separation is motivated by the duplex-sponge Fiat–Shamir transformation of Chiesa and Orru and by the practical difficulty of maintaining transcript discipline across different protocols, examples, and compatibility paths.

The project supports interactive arguments, interactive reductions, indexed preprocessing, composition, instance-aware security metadata, a live interactive backend, and a non-interactive backend based on spongefish's DSFS compiler. The main result is an interface layer that keeps protocol implementations free of sponge operations while still supporting non-trivial case studies, including Schnorr, sumcheck-style examples, a compatibility bridge for sigma protocols, and WARP as a preprocessing interactive reduction composed with a final decider. Argus is not a production-reviewed cryptographic library; it is a prototype for protocol design and security-model engineering.

1 Motivation

Many modern proof systems begin life as public-coin interactive protocols. A prover sends a commitment or algebraic message, the verifier samples a public challenge, and the protocol repeats until the verifier either accepts or computes a reduced target claim. In implementations, these protocols are often made non-interactive with Fiat–Shamir: prover messages are absorbed into a transcript, verifier challenges are derived from that transcript, and the resulting transcript string becomes a proof.

This implementation pattern creates a persistent engineering problem. The mathematical protocol is usually described round by round, but the code must also enforce a low-level transcript discipline. Every prover message must be absorbed before the next challenge is squeezed. Public inputs, domain-separation data, session identifiers, and indexed-relation commitments must be fixed before the first challenge. Verification must replay the same operations deterministically, and it must reject malformed proof bytes rather than silently accepting a prefix. These requirements are easy to state and easy to violate, especially when each protocol implementation manually handles its own transcript.

Argus addresses this by treating the transcript as a backend concern. A protocol author writes an interactive protocol against a generic channel API. The same protocol can then be executed by a live interactive backend, where verifier messages are sampled and sent over channels, or by a duplex-sponge Fiat–Shamir backend, where verifier messages are squeezed from a transcript and prover messages are serialized into a non-interactive argument. The channel API is also a target language: a compiler such as a future IOP-to-IA or iBCS compiler can output the same kind

of channel program before DSFS turns it into a non-interactive argument. The author does not instantiate a sponge, absorb public input, or derive challenges manually. The protocol code is intended to read like the interactive protocol, while the backend enforces the transcript invariants.

The second motivation is reductions. Many proof systems are not naturally just "accept or reject" protocols. An interactive oracle reduction, folding scheme, or accumulation scheme may transform one claim into another claim. The verifier's output is a target instance, not a Boolean. Modeling only interactive arguments forces these protocols into awkward encodings. Argus therefore has first-class interactive reductions alongside interactive arguments, and it supports composing reductions with other reductions or with a final argument.

The third motivation is preprocessing. Protocols such as WARP have static relation material, code parameters, Merkle parameters, and circuit descriptions that should be bound before any challenge, but should not be repeatedly copied into each per-claim instance. Argus models this with preprocessing keys. A preprocessing protocol derives a prover key and verifier key from an index; the compiled backend receives the relevant key as an input and absorbs a committed index together with the ordinary instance. This keeps the non-interactive wrapper stateless while still binding the indexed relation into the transcript.

2 Design Objectives and Constraints

The primary objective is a clean channel interface that can be used both by protocol authors and by protocol compilers. Protocol implementations should depend on the interface crate, not on spongefish transcript internals. On the prover side, the available operations are sending a prover message and reading a verifier message. On the verifier side, the available operations are reading a prover message and sending a verifier message. This is deliberately small: it prevents protocol code from deciding where absorb and squeeze operations occur, and it gives higher-level compilers a concrete IA target to emit.

The second objective is backend-owned transcript semantics. In the DSFS backend, sending a prover message appends it to the proof string and absorbs it into the sponge. Reading a verifier message squeezes the next challenge. During verification, reading a prover message consumes proof bytes and absorbs the same message before the verifier derives the matching challenge. This ownership boundary is what protects the key invariants: public inputs before the first challenge, all prover messages before a challenge, deterministic replay, no challenge reuse, and no hidden transcript state outside the backend.

The third objective is to support both interactive arguments and interactive reductions. An interactive argument has a public instance, a private witness, and a verifier that returns accept or reject. An interactive reduction has a source instance and witness, and it produces a target instance and target witness on the prover side while the verifier computes the target instance. This distinction matters because it gives accumulation and folding-style protocols a natural representation.

The fourth objective is composability. A reduction followed by another reduction is again a reduction, and a reduction followed by an argument is an argument for the original relation. Argus exposes this through composition adapters. Composition also has to preserve domain separation: composed protocol identifiers are derived using structured, length-prefixed encodings rather than ambiguous string concatenation.

The fifth objective is instance-aware security metadata. For protocols such as sumcheck or WARP, the soundness profile depends on instance-derived parameters: field size, degree bounds, number of rounds, code length, constraint count, or query counts. Argus therefore does not attach

a single global security number to a protocol type. Instead, security traits compute profiles for concrete instances or for explicit worst-case instance families. The DSFS layer can then combine an interactive profile with sponge parameters and adversarial query budgets.

The main cryptographic constraint is that Argus should not duplicate transcript logic across modules. Sponge operations belong in DSFS. Public input absorption belongs in the backend, including protocol identifiers, sponge information, sessions, ordinary instances, and preprocessing committed-index bytes. Protocol code may branch on public messages and public challenges, but it must not derive Fiat–Shamir challenges itself. If sponge choice, salt policy, proof layout, or absorb/squeeze order changes, the change is a backend-level change and requires a protocol-identifier review.

There is also a compatibility constraint. The project uses Keccak as the Argus standard DSFS sponge. However, some compatibility paths, especially the sigma-proofs bridge, need spongefish's `StdHash` construction based on SHAKE128 to match external vector formats. The interface must allow this without making SHAKE128 the default Argus transcript. This is why the report and documentation distinguish the Argus standard from spongefish's "standard hash" terminology.

3 Architecture

Argus is organized as a Rust workspace. The `ia-core` crate is the protocol-facing layer. It defines channel traits, protocol core traits, interactive argument and reduction traits, preprocessing traits, non-interactive vocabulary, composition adapters, and security metadata. The live backend is implemented in `live-channel`. The DSFS backend is provided by the local `spongefish::dsfs` dependency. Protocol examples and demonstrations live in `argus-examples`; WARP lives in `warp`; and sigma-proofs compatibility lives in `sigma-bridge`. IOP-to-IA compilation remains a future direction rather than a workspace crate, but the interface is designed so such a compiler can emit the same channel programs that hand-written protocols use.

The core trait tree separates identity, relation shape, and execution. Every protocol implements `ProtocolCore`, which supplies a protocol identifier for domain separation. Arguments implement `ArgumentCore`, which defines the public instance and private witness types. Reductions implement `ReductionCore`, which defines source and target instances and witnesses. The executable leaves are `InteractiveArgument` and `InteractiveReduction`. Preprocessing protocols also implement `PreprocessingCore`, which defines the index, prover key, verifier key, and deterministic preprocessing function.

The channel traits are the narrow waist of the system. A prover channel has an associated alphabet `Unit`; byte-oriented protocols use `Unit = u8`, while the type is general enough for algebraic sponge settings. Prover messages implement the encoding and serialization traits needed to be written into a non-interactive proof. Verifier messages implement decoding. These bounds are intentionally close to spongefish's codec vocabulary, which made it possible to reuse the existing DSFS machinery. One design question left for future cleanup is whether `ia-core` should own a smaller codec abstraction so the abstract channel layer is less visibly tied to the DSFS implementation.

The DSFS constructors are semantic: they name the object being constructed. The plain argument constructor compiles an interactive argument into a non-interactive argument, and the plain reduction constructor compiles an interactive reduction into a non-interactive reduction. Preprocessing variants return stateless compiled handles. The caller runs preprocessing to obtain a prover key and verifier key, then passes the relevant key to proving or verification. The compiled object stores the protocol body and sponge choice, not the keys.

Preprocessing is represented as "keys as inputs." Both prover and verifier keys implement

`CommittedIndex`. The prover-side compiled path binds `pk.committed_index()`, while the verifier-side path binds `vk.committed_index()`. The backend absorbs this committed index together with the per-claim instance before deriving any verifier challenge. This design is important for indexed relations: the verifier can bind the static relation without the prover needing to hold the verifier key, and the ordinary instance remains a per-claim object rather than a container for static setup material.

The live backend gives the same channel program a genuine interactive execution. The verifier samples messages and sends them to the prover over Rust channels. This backend is not meant to be a network protocol. Its value is as a sanity check: if a protocol cannot be expressed as public-coin channel code, it should not be silently compiled through DSFS.

4 Representative Implementations

The examples crate demonstrates the authoring model on small protocols. Schnorr can be run non-interactively through DSFS or interactively through the live backend. Sumcheck examples exercise multi-round public-coin structure. A committed sumcheck example includes Merkle commitments and query openings. A composition example shows how reductions and arguments can be connected without manually reimplementing the combined transcript.

The sigma bridge tests compatibility with an external protocol library. It drives sigma-proofs protocols through the Argus IA-to-DSFS pipeline. This part of the project is intentionally geared toward compatibility: it preserves expected session derivation behavior and uses `StdHash` only for explicit SHAKE128 vector matching. In particular, `derive_session_id` uses SHAKE128 with the domain `fiat-shamir/session-id` and writes output into the high 32 bytes of the 64-byte session field. The bridge is a useful stress test because it forces the Argus abstraction to interact with existing proof formats rather than only toy examples.

The largest case study is WARP, a linear-time accumulation scheme for R1CS. In the paper, WARP is naturally an interactive oracle reduction: it reduces a set of fresh and accumulated claims into a single accumulated claim, then a decider checks the accumulated witness. Argus models this as a preprocessing interactive reduction plus a preprocessing interactive argument. The `WarpReduction` performs the reduction and outputs accumulator instances; the `WarpDecider` performs the final local check; and `FullWarp` exposes a single-index preprocessing argument for the common end-to-end path.

This case study exercised the most important design constraints. Static WARP material, including dimensions, configuration, code material, Merkle parameters, and R1CS matrices, is part of the preprocessing index. The derived verifier key commits to this material. The per-claim WARP instance contains only fresh instances and previous accumulators. DSFS absorbs the committed index and the per-claim instance before any WARP challenge. The WARP protocol engine itself uses only channel operations for messages and challenges; it does not own the sponge. This is precisely the separation Argus was built to enforce.

The WARP security implementation is instance-aware. The reduction derives security parameters from the static index, including round counts, field size, code parameters, and WARP configuration. It reports per-round round-by-round soundness terms for the twin sumcheck and batching sumcheck phases, plus one-time commitment and sampling terms. Some code-specific bounds are currently conservative, and the precise placement of certain commitment-phase terms is documented as a future review point rather than hidden as a settled theorem.

5 Results

The main result of the project is an executable separation between protocol logic and transcript logic. Protocol implementations use a small channel API. DSFS owns absorb/squeeze ordering, public-input binding, proof serialization, and deterministic replay. The live backend owns interactive transport. This means that a protocol such as Schnorr can be authored once and then run in both modes, and a more involved reduction such as WARP can be compiled non-interactively without embedding sponge calls in the protocol code.

The second result is a unified interface for arguments, reductions, and preprocessing. Earlier designs often become strained when reductions enter the picture, because the verifier output is not a Boolean. Argus makes that output part of the type system. Composition adapters can therefore express `IR -> IR` and `IR -> IA` composition directly. Preprocessing also fits the same structure: keys are explicit inputs, compiled wrappers are stateless, and committed-index bytes are absorbed by the backend as public input.

The third result is a security metadata layer that matches the way protocols are parameterized in practice. Instead of a single protocol-level error bound, Argus supports profiles derived from concrete instances or explicit instance families. This is especially useful for reductions because the target-instance bound of one reduction can feed the profile of the next protocol in a composed chain.

The fourth result is a set of regression tests and examples that exercise the architecture. The WARP integration tests check deterministic index commitments, commitment sensitivity to relation/code/configuration changes, rejection under the wrong verifier key, separation between static index material and per-claim instances, DSFS prove/verify paths for the reduction, and the single-index `FullWarp` path. The sigma bridge has golden-vector tests for SHAKE128 compatibility. The examples crate provides runnable DSFS and live-channel protocols.

From a project-submission perspective, Argus is best presented as a systems and interface contribution rather than as a new cryptographic construction. It implements and enforces a disciplined boundary around an analyzed DSFS transformation, and it demonstrates that the boundary is expressive enough for small arguments, reductions, preprocessing, external compatibility, and a WARP case study.

6 Limitations and Future Work

Argus remains a research prototype. It has not received production cryptographic review. The code is intended for protocol experimentation and security-model engineering, not deployment. This distinction is important because transcript correctness is necessary for security but not sufficient for a production proof system.

The long-term architectural goal is still to express BCS as DSFS applied to an interactive argument obtained from an IOP and a Merkle-tree commitment scheme. Equivalently, the channel layer should be the output of an iBCS-style compiler as well as the input to DSFS. The current project lays the interface groundwork for that direction, but it does not include an IOP-to-IA compiler.

The channel codec boundary could be improved. Today, `ia-core` re-exports spongefish encoding and serialization vocabulary. This choice kept the prototype small and compatible with the DSFS backend, but it also means the abstract protocol-facing crate exposes names that are historically tied to the non-interactive backend. A cleaner future design may define a smaller Argus codec layer and adapt it to spongefish at the backend boundary.

The WARP security profile is useful but not final. Some list-size and commitment-phase terms are intentionally conservative. Tightening them requires a closer pass against the WARP analysis and the precise adversarial model used by the DSFS security helper. The current implementation makes those choices visible in documentation and tests rather than presenting them as completed formalization.

Finally, compatibility paths need continued care. The sigma bridge demonstrates that Argus can match external layouts when explicitly configured, but this should not blur the default transcript story. Keccak remains the Argus standard DSFS sponge, while SHAKE128 `StdHash` is a compatibility selection. Any future change to sponge choice, proof layout, salt policy, or transcript domain separation must trigger a protocol-identifier review.

7 Conclusion

Argus shows that public-coin protocol implementations can be written as clean channel programs while leaving transcript mechanics to execution backends. This is a small interface idea with large consequences: it reduces duplicated Fiat–Shamir logic, makes transcript invariants local to the DSFS compiler, supports genuine interactive execution for sanity checks, and gives reductions and preprocessing a first-class place in the API.

The project's contribution is not a new proof system. It is a disciplined Rust architecture for implementing proof-system components without mixing protocol logic with transcript internals. The case studies suggest that the abstraction is expressive enough for both simple examples and a substantial accumulation protocol. The remaining work is to sharpen the codec boundary, complete the IOP-to-IA direction, and continue validating the security metadata against the underlying papers.

References

[1] Alessandro Chiesa and Michele Orru. *A Fiat-Shamir Transformation From Duplex Sponges*. IACR ePrint 2025/536, 2025. `https://eprint.iacr.org/2025/536`

[2] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. *Linear-Time Accumulation Schemes*. IACR ePrint 2025/753, 2025. `https://eprint.iacr.org/2025/753`

[3] spongefish and spongefish-dsfs. Rust implementation of the duplex-sponge Fiat–Shamir transformation used by Argus. `https://github.com/ricardo-perello/spongefish`